LEASEGUARD

Minimum Viable Product (MVP)

Product Requirements and Technical Development Specification

PREPARED FOR	Nexus Solution
PROJECT NAME	LeaseGuard
DOCUMENT TYPE	MVP Product Requirements Document (PRD)
VERSION	1.0
DATE	31 October 2026
STATUS	Proposed Development Specification

Table of Contents

1. PROJECT OVERVIEW	3
2. PROBLEM STATEMENT	4
3. PRODUCT OBJECTIVES	5
4. TARGET USERS AND ROLES	5
5. MVP FUNCTIONAL MODULES	6
6. WHATSAPP BUSINESS INTEGRATION	13
7. WEB APPLICATION REQUIREMENTS	16
8. NON-FUNCTIONAL REQUIREMENTS	18
9. PROPOSED TECHNOLOGY STACK	19
10. SYSTEM ARCHITECTURE	22
11. DATABASE DESIGN	23
12. DEVELOPMENT TIMELINE	24
13. MVP TESTING AND ACCEPTANCE CRITERIA	28
14. PILOT IMPLEMENTATION PLAN	29
15. SECURITY AND DATA PROTECTION	30
16. MVP OUT OF SCOPE	31
17. FUTURE ENHANCEMENTS	32
18. PROJECT RISKS AND MITIGATION	33
19. PROJECT DEPENDENCIES AND ASSUMPTIONS	33
20. FINAL MVP DELIVERABLES	34
21. FINAL DEVELOPMENT RECOMMENDATION	34

1. PROJECT OVERVIEW

1.1 Introduction

LeaseGuard is a commercial property lease monitoring and tenant communication platform designed to help property owners and property managers monitor lease obligations, identify financial discrepancies and automate administrative communication.

The platform works alongside the client's existing accounting software. It does not replace existing accounting systems or require direct integration with them during the MVP.

LeaseGuard receives PDF statements generated by the client's accounting software, extracts relevant information and compares the billed amounts against the terms recorded in the corresponding lease agreements.

The system automatically monitors important lease-related events, including:

- Rental escalation discrepancies.
- Lease expiry dates.
- Renewal option deadlines.
- Deposit and bank guarantee amounts.
- Bank guarantee expiry dates.
- Outstanding tenant balances.
- Arrears.
- Payment arrangements recorded by property managers.

LeaseGuard will consist of two primary interfaces:

1. **Web Application:** Used by property managers to manage properties, tenants, leases, statements, monitoring events and communication.
2. **WhatsApp Business Integration:** Used to communicate with tenants and property owners through each client's own approved WhatsApp Business number.

The primary objective is to reduce the financial and operational risks associated with missed lease obligations, incorrect rental billing and inconsistent tenant follow-ups.

1.2 Core Business Principle

LeaseGuard is strictly a monitoring and communication platform.

> **LeaseGuard will not process, collect, receive, hold or facilitate rental payments.**

Tenants will continue paying property owners or landlords directly through their existing payment channels.

The platform will only:

- Read rental information from uploaded statements.
- Compare statement information against lease agreements.
- Identify outstanding balances.

- Monitor arrears.
- Record payment arrangements agreed between property managers and tenants.
- Send approved payment reminders.
- Track communication history.

There will be no payment gateway, banking integration, transaction processing or payment collection functionality.

2. PROBLEM STATEMENT

Commercial property owners and property managers often manage portfolios containing hundreds of tenants and lease agreements.

Each lease contains important financial obligations and deadlines, including:

- Monthly rental amounts.
- Annual rental escalations.
- Lease expiry dates.
- Renewal option periods.
- Deposit requirements.
- Bank guarantee requirements.
- Payment obligations.

This information is frequently stored across PDF documents, spreadsheets, accounting systems and manual records.

Although accounting software records the amounts billed to tenants, it does not necessarily verify whether those amounts correspond with the original lease agreements.

As a result, several problems can occur:

1. Annual rental escalations may not be applied correctly.
2. Leases may expire without sufficient preparation.
3. Renewal option deadlines may be missed.
4. Bank guarantees may expire or become insufficient.
5. Outstanding tenant balances may not be followed up consistently.
6. Payment arrangements may not be properly monitored.
7. Property managers may spend significant time manually checking statements and lease documents.
8. Property owners may lack a clear and consistent overview of portfolio performance.

LeaseGuard addresses these challenges by centralising lease information, automating monitoring activities and providing timely notifications to the relevant people.

3. PRODUCT OBJECTIVES

The MVP must achieve the following objectives:

1. Provide a centralised web application for managing commercial property information.
2. Support multiple client companies within one platform.
3. Allow property managers to manage properties, units, tenants and lease agreements.
4. Allow users to upload PDF statements generated by existing accounting systems.
5. Extract relevant information from uploaded statements.
6. Compare extracted statement information against recorded lease terms.
7. Automatically identify lease discrepancies and upcoming obligations.
8. Provide property managers with a daily action dashboard.
9. Automate approved tenant statement delivery and payment reminders through WhatsApp.
10. Provide property owners with weekly portfolio summaries through WhatsApp.
11. Allow property owners to ask predefined or supported portfolio-related questions through WhatsApp.
12. Maintain a complete history of monitoring events and communication.
13. Ensure each client company communicates using its own approved WhatsApp Business number.

4. TARGET USERS AND ROLES

The MVP will support three main user categories.

4.1 Property Owner

The property owner receives important portfolio information through WhatsApp.

The owner should not need to log into the web application for routine reporting.

Capabilities:

- Receive weekly portfolio summaries.
- View total outstanding tenant balances.
- Receive important portfolio updates.
- Identify properties requiring attention.
- Ask questions about outstanding balances.
- Ask which leases are approaching expiry.
- Receive notifications about significant monitoring events.

4.2 Property Manager

The property manager is the primary user of the web application.

Capabilities:

- Access the company dashboard.
- Create and manage properties.

- Create and manage units.
- Manage tenant information.
- Capture and update lease information.
- Upload PDF statements.
- Review extracted statement information.
- Review rental discrepancies.
- Monitor lease expiry and renewal deadlines.
- Monitor bank guarantees.
- Monitor tenant arrears.
- Record payment arrangements.
- Approve tenant communication.
- View WhatsApp message history.
- Mark monitoring events as completed.

4.3 Tenant

Tenants interact with LeaseGuard through WhatsApp.

Tenants may receive:

- Monthly statements.
- Initial payment reminders.
- Overdue payment reminders.
- Follow-up reminders.
- Payment arrangement instalment reminders.

Tenants must not receive automated communication about rental escalations, lease expiries or renewal options.

These matters must remain under the control of the property manager.

5. MVP FUNCTIONAL MODULES

5.1 Module 1: Authentication and Company Management

Objective: Provide secure access to the platform and support multiple client companies.

Functional requirements:

- User login and logout.
- Secure authentication.
- Company registration and configuration.
- Company-specific workspaces.
- User invitation and management.
- Role-based access control.

- Company-specific WhatsApp configuration.
- User account management.
- Audit logging.

Each client company must have access only to its own information.

A user associated with one company must not be able to access another company's records unless explicitly authorised.

5.2 Module 2: Property Management

Objective: Allow property managers to manage their commercial property portfolios.

Functional requirements:

- Create properties.
- Edit property information.
- View property details.
- Archive properties.
- Associate properties with a client company.
- View the units belonging to a property.
- View property-level monitoring information.

Property information:

- Property name.
- Property address.
- Property type.
- Property description.
- Company association.
- Property status.

5.3 Module 3: Unit Management

Objective: Manage individual commercial units within each property.

Functional requirements:

- Create units.
- Edit unit details.
- Associate units with properties.
- Assign tenants to units.
- Associate leases with units.
- Track unit occupancy status.

Unit information:

- Unit number.
- Property.

- Unit type.
- Occupancy status.
- Associated tenant.
- Associated lease.

5.4 Module 4: Tenant Management

Objective: Maintain accurate tenant records for lease monitoring and communication.

Functional requirements:

- Create tenant records.
- Update tenant information.
- Associate tenants with units.
- Associate tenants with leases.
- Store WhatsApp contact numbers.
- Store email addresses where available.
- View tenant statement history.
- View tenant arrears information.
- View tenant communication history.

Tenant information:

- Tenant name or registered business name.
- Contact person.
- WhatsApp number.
- Email address.
- Associated property.
- Associated unit.
- Lease reference.
- Tenant status.

WhatsApp numbers must be validated before automated communication is enabled.

5.5 Module 5: Lease Management

Objective: Store and manage the contractual information required by the monitoring engine.

Functional requirements:

- Create lease records.
- Edit lease information.
- Associate leases with tenants.
- Associate leases with properties and units.
- Upload lease documents.
- Capture rental amounts.

- Capture escalation conditions.
- Capture lease expiry dates.
- Capture renewal option information.
- Capture deposit information.
- Capture bank guarantee information.
- Track lease status.

Required lease information:

Field	Description
Lease reference	Unique lease identifier
Tenant	Associated tenant
Property	Associated property
Unit	Associated unit
Commencement date	Lease start date
Expiry date	Lease end date
Monthly rental	Contractual rental amount
Escalation percentage	Agreed rental increase
Escalation frequency	Annual or other agreed interval
Escalation date	Date the increase becomes effective
Renewal option	Whether a renewal option exists
Renewal notice period	Required notice period
Deposit amount	Agreed deposit
Guarantee amount	Bank guarantee value
Guarantee expiry	Guarantee expiry date

Lease terms must be verified by an authorised person before they are used for financial monitoring.

Automatic lease document extraction may be considered in a later phase.

5.6 Module 6: PDF Statement Upload and Processing

Objective: Extract relevant information from statements generated by the client's existing accounting software.

LeaseGuard should support PDF statements from different accounting systems without requiring direct integration.

Potential source systems include:

- MRI.
- MDA.
- PropSys.
- Sage.
- Other systems capable of generating PDF statements.

Functional requirements:

- Upload individual PDF statements.
- Upload multiple statements where supported.
- Associate statements with a property or processing batch.
- Store original PDF documents.
- Extract relevant statement information.
- Identify tenants and units.
- Validate extracted financial information.
- Flag uncertain extraction results.
- Allow manual corrections.
- Prevent accidental duplicate processing.
- Maintain processing history.

Information to Extract

- Tenant name.
- Unit number.
- Statement date.
- Statement period.
- Rental charges.
- Additional charges.
- Payments reflected on the statement.
- Outstanding balance.
- Total amount due.

Processing Workflow

1. An authorised user uploads a PDF statement.
2. The document is stored securely.
3. The document-processing service extracts the text and relevant information.
4. Extracted information is converted into structured data.
5. Validation checks are performed.
6. The system attempts to match the statement with the correct tenant and unit.
7. Uncertain information is flagged.
8. The property manager reviews and confirms the information.
9. Verified information is stored in the database.

10. The monitoring engine evaluates the statement against the lease.

The original PDF must remain available for reference.

The system must not silently guess missing financial information.

5.7 Module 7: Lease Monitoring Engine

Objective: Automatically identify discrepancies and upcoming lease obligations.

This module represents the core business functionality of LeaseGuard.

The monitoring engine should use predefined and explainable business rules.

It should run automatically on a scheduled basis and whenever relevant verified information is updated.

A. Rental Escalation Monitoring

The system must calculate the expected rental amount using the escalation conditions recorded in the lease.

It must compare the expected rental against the amount reflected in the uploaded statement.

Example:

- Original monthly rental: R35,000.
- Annual escalation: 8%.
- Expected rental after escalation: R37,800.
- Actual billed rental: R35,000.
- Identified monthly discrepancy: R2,800.

The system must:

- Detect the discrepancy.
- Record the expected amount.
- Record the billed amount.
- Calculate the difference.
- Display the calculation.
- Create a monitoring event.
- Notify the property manager.

The system must not automatically communicate rental increases to tenants.

B. Lease Expiry Monitoring

The system must monitor lease expiry dates.

Functional requirements:

- Calculate upcoming expiry dates.
- Identify leases approaching expiry.
- Create advance notifications.
- Display the affected tenant and property.

- Assign the event to the responsible property manager.
- Track the event status.

Notification intervals should be configurable.

C. Renewal Option Monitoring

The system must monitor renewal option periods and calculate notice deadlines.

Example:

If a lease expires on 31 December and the tenant must exercise a renewal option three months before expiry, the system should calculate the applicable deadline.

Functional requirements:

- Identify leases containing renewal options.
- Calculate notice deadlines.
- Generate advance warnings.
- Display the relevant lease information.
- Notify the property manager.

The system must never automatically inform tenants about upcoming renewal options.

D. Deposit and Bank Guarantee Monitoring

The system must monitor deposits and bank guarantees.

Functional requirements:

- Store deposit amounts.
- Store guarantee amounts.
- Store guarantee expiry dates.
- Identify guarantees approaching expiry.
- Compare guarantee coverage against the required amount.
- Identify shortfalls.
- Generate monitoring events.

Example:

If a guarantee was originally based on three months of rental, the system should calculate whether the current guarantee still covers the agreed amount after rental escalations.

E. Arrears Monitoring

The system must identify tenants with outstanding balances using uploaded statements.

Functional requirements:

- Identify outstanding tenant balances.
- Categorise arrears by age.
- Display overdue amounts.
- Identify tenants requiring follow-up.
- Generate monitoring events.

- Support automated payment reminder scheduling.

Arrears information must be based on verified statement information.

F. Payment Arrangement Monitoring

> **This module records agreements. It does not process payments.**

Property managers must be able to record payment arrangements agreed directly between the tenant and the property owner or manager.

Required information:

- Tenant.
- Property.
- Outstanding balance.
- Agreed instalment amount.
- Instalment frequency.
- First instalment date.
- Final instalment date.
- Arrangement status.
- Notes.

Functional requirements:

- Create an arrangement.
- Edit an arrangement.
- Pause normal arrears reminders while an arrangement is active.
- Schedule instalment reminders.
- Identify missed instalment dates using updated statement information or manager confirmation.
- Notify the property manager when an arrangement requires attention.
- Close or cancel an arrangement.

LeaseGuard does not collect instalments or verify bank transactions.

6. WHATSAPP BUSINESS INTEGRATION

6.1 Overview

WhatsApp Business is a core component of LeaseGuard.

The platform must integrate with the official Meta WhatsApp Business Platform Cloud API.

The system must support multiple client companies, with each company communicating through its own approved WhatsApp Business number.

Unofficial WhatsApp Web automation must not be used.

6.2 Company-Specific WhatsApp Accounts

Each client company must have its own WhatsApp Business configuration.

Example:

- Client Company A sends messages through Company A's WhatsApp Business number.
- Client Company B sends messages through Company B's WhatsApp Business number.
- Client Company C sends messages through Company C's WhatsApp Business number.

The platform must select the correct WhatsApp account based on the company associated with the relevant tenant or property owner.

Required configuration:

- Company ID.
- WhatsApp Business Account ID.
- Phone Number ID.
- Business display name.
- Secure access token reference.
- Webhook configuration.
- Approved message templates.
- Connection status.

Credentials must be securely stored and must never be exposed to frontend code.

Each client must complete the relevant Meta onboarding, business verification and number registration requirements.

6.3 Tenant Communication

Tenant communication must be limited to statements and payment-related reminders.

Supported messages:

1. Monthly statement delivery.
2. Initial payment reminder.
3. Overdue payment reminder.
4. Follow-up payment reminder.
5. Payment arrangement instalment reminder.

The system must not automatically send messages about:

- Rental escalations.
- Lease expiries.
- Renewal options.
- Renewal negotiations.
- Disputed charges.
- Guarantee-related decisions.

These matters remain under the control of the property manager.

Statement Delivery

The system should send the original statement as a WhatsApp document attachment where appropriate.

Bank account details must only appear on the original statement attachment and must not be typed into WhatsApp message text.

Tenants should be informed to expect messages from the relevant client's verified WhatsApp Business number.

6.4 Owner Communication

Property owners must receive a weekly portfolio summary through WhatsApp.

The recommended schedule is every Monday.

The summary should include:

- Total outstanding balances.
- Changes since the previous report.
- Important lease monitoring events.
- Significant rental discrepancies.
- Expiring guarantees.
- Items requiring the owner's attention.

Owners should also be able to ask supported portfolio-related questions.

Examples:

- What is the total outstanding balance?
- Which tenants are overdue?
- What is outstanding at a particular property?
- Which leases expire next year?
- What changed in the portfolio this week?

The system must only answer using authorised and verified company data.

If information is unavailable or uncertain, the system must communicate that rather than inventing an answer.

6.5 Incoming Messages and Webhooks

The integration must support incoming WhatsApp messages and message status notifications.

Functional requirements:

- Receive incoming messages.
- Identify the sender.
- Associate the sender with the correct company.
- Process supported responses.
- Record outgoing message statuses.
- Record incoming replies.

- Handle failed messages.
- Handle duplicate webhook events safely.
- Maintain a complete communication history.

The system must also support appropriate opt-out and message suppression handling.

6.6 WhatsApp Templates and Approval

The platform must support approved WhatsApp message templates.

Business-initiated messages must follow the applicable Meta messaging rules.

The system must respect the applicable 24-hour customer service window.

During the MVP pilot, tenant communication should require approval from an authorised property manager before messages are sent.

No unapproved tenant communication should be sent.

7. WEB APPLICATION REQUIREMENTS

The web application must provide a central dashboard and dedicated management screens.

Screen	Functionality
Login	Secure user authentication
Dashboard	Portfolio overview and important actions
Properties	Property management
Units	Unit management
Tenants	Tenant management
Leases	Lease creation and management
Statements	PDF statement uploads and history
Statement Review	Extracted data validation
Monitoring Events	Discrepancies and upcoming obligations
Payment Arrangements	Record and monitor agreed arrangements
WhatsApp Messages	Incoming and outgoing message history
WhatsApp Settings	Company-specific WhatsApp configuration
User Management	Manage company users and permissions
Audit Logs	View important system activity

7.1 Dashboard Information

The dashboard should display:

- Total properties.
- Total units.
- Occupied units.
- Total active leases.
- Total outstanding balances.
- Rental escalation discrepancies.
- Upcoming lease expiries.
- Upcoming renewal deadlines.
- Expiring bank guarantees.
- Tenants in arrears.
- Active payment arrangements.
- Pending statement reviews.
- Pending message approvals.

7.2 Monitoring Event Management

Each monitoring event should contain:

- Event ID.
- Event type.
- Company.
- Property.
- Unit.
- Tenant.
- Lease reference.
- Description.
- Date identified.
- Priority.
- Recommended action.
- Assigned property manager.
- Due date.
- Status.

Possible event statuses:

- New.
- Under Review.
- Action Required.
- In Progress.
- Resolved.

- Dismissed.

Dismissed events should retain a reason and audit history.

8. NON-FUNCTIONAL REQUIREMENTS

8.1 Security

- Secure authentication.
- Role-based access control.
- Company-level data isolation.
- Protected document storage.
- Secure API communication.
- Encrypted sensitive credentials.
- Input validation.
- Audit logging.
- Restricted administrative access.

8.2 Reliability

- Background jobs must support retries.
- Failed jobs must be recorded.
- Duplicate messages must be prevented.
- Webhook events must be processed safely.
- Important processing errors must be visible to authorised users.

8.3 Performance

- Dashboard information should load efficiently.
- Statement processing should run asynchronously.
- Long-running document extraction must not block normal application usage.
- Monitoring jobs should be scheduled and processed in the background.

8.4 Scalability

The architecture must support additional client companies, properties, units, tenants and statements without requiring a complete redesign.

8.5 Auditability

The system must record:

- User actions.
- Statement uploads.
- Statement corrections.
- Lease changes.
- Monitoring events.
- Message approvals.
- Messages sent.
- Message delivery statuses.
- Payment arrangement changes.

9. PROPOSED TECHNOLOGY STACK

The following technology stack is recommended for building an efficient and maintainable MVP.

The preferred approach is a modular monolithic application with a separate document-processing service and background worker.

9.1 Frontend

Next.js + React + TypeScript

Purpose:

- Build the web application.
- Create dashboards.
- Manage navigation.
- Build property management screens.
- Build forms and tables.
- Communicate with backend services.

Supporting tools:

- Tailwind CSS.
- shadcn/ui.
- Lucide icons.
- React Hook Form.
- Zod.

9.2 Main Backend

Next.js Server-Side Functions and API Routes

The main application backend will use TypeScript.

Responsibilities:

- Authentication and authorisation.
- Company management.
- Property management.
- Unit management.
- Tenant management.
- Lease management.
- Statement management.
- Monitoring event management.
- WhatsApp integration.
- Dashboard reporting.
- Audit logging.

9.3 Database

PostgreSQL through Supabase

Supabase will provide:

- Managed PostgreSQL database.
- Authentication.
- File storage.
- Row Level Security.
- Database management.

PostgreSQL is recommended because the platform contains structured relationships between companies, properties, units, tenants, leases and statements.

9.4 Document Processing

Python + FastAPI

The document-processing service will handle PDF statement extraction.

Recommended libraries:

- PyMuPDF.
- pdfplumber.
- Tesseract OCR.
- Pydantic.

Responsibilities:

- Receive PDF documents.
- Extract text and tables.
- Identify statement information.
- Convert extracted data into structured records.
- Validate extracted information.
- Flag uncertain results.

9.5 Background Processing

Redis + BullMQ

Used for scheduled and asynchronous operations.

Examples:

- Nightly lease monitoring.
- Statement processing.
- Tenant payment reminders.
- Instalment reminders.
- Weekly owner reports.
- WhatsApp message retries.
- Webhook processing.

9.6 WhatsApp Integration

Meta WhatsApp Business Platform Cloud API

Used for official WhatsApp communication.

Responsibilities:

- Send tenant statements.
- Send approved payment reminders.
- Send weekly owner summaries.
- Receive replies.
- Receive delivery status updates.
- Support multiple client WhatsApp numbers.

9.7 Testing

- Vitest for unit testing.
- Playwright for end-to-end browser testing.
- Pytest for Python document processing.
- API integration tests.

9.8 Hosting and Deployment

Component	Recommended Service
Frontend	Vercel
Main backend	Vercel server-side runtime
Database	Supabase PostgreSQL
Authentication	Supabase Auth
File storage	Supabase Storage
PDF processing	Render or equivalent
Background worker	Render or equivalent
Redis	Managed Redis service
Source control	GitHub
CI/CD	GitHub Actions
WhatsApp	Meta Cloud API

10. SYSTEM ARCHITECTURE

The proposed architecture consists of five main layers.

10.1 Presentation Layer

Technology: Next.js, React and TypeScript.

Provides the web interface for property managers and authorised company users.

10.2 Application Layer

Technology: TypeScript and Next.js server-side functions.

Handles business logic, permissions, data validation, API operations and communication between system components.

10.3 Data Layer

Technology: Supabase PostgreSQL and Supabase Storage.

Stores company information, properties, units, tenants, leases, statements, monitoring events and message records.

10.4 Processing Layer

Technology: Python, FastAPI, Redis and BullMQ.

Responsible for document extraction, monitoring jobs and scheduled automation.

10.5 Communication Layer

Technology: Meta WhatsApp Business Cloud API.

Responsible for tenant and owner communication using the correct client company's WhatsApp Business number.

10.6 End-to-End Workflow

1. Property manager uploads monthly statements.
2. The system securely stores the documents.
3. The Python service extracts relevant information.
4. The system validates the extracted information.
5. Uncertain information is sent for manual review.
6. Verified data is stored in PostgreSQL.
7. The monitoring engine compares statement information against lease terms.
8. Monitoring events are generated.
9. Property managers review findings.
10. Approved tenant messages are sent through the correct company's WhatsApp Business number.
11. Message history and delivery statuses are recorded.
12. Owners receive weekly portfolio summaries through WhatsApp.

11. DATABASE DESIGN

The following database entities are recommended.

Table	Purpose
companies	Stores client company records
users	Stores user accounts
company_users	Connects users to companies and roles
whatsapp_accounts	Stores company-specific WhatsApp configurations
whatsapp_templates	Stores approved message templates
properties	Stores commercial property information
units	Stores individual property units
tenants	Stores tenant information

Table	Purpose
leases	Stores contractual lease information
lease_documents	Stores lease document references
statements	Stores uploaded statement records
statement_entries	Stores extracted statement information
monitoring_events	Stores lease discrepancies and upcoming events
payment_arrangements	Stores agreed payment arrangement records
messages	Stores incoming and outgoing WhatsApp messages
message_approvals	Stores message approval decisions
job_runs	Tracks background jobs
audit_logs	Stores important system activity

All company-owned records must contain a company identifier to support tenant isolation and access control.

Database clarification: Statement balances and payment arrangement information are monitoring records. They do not represent transactions processed by LeaseGuard.

12. DEVELOPMENT TIMELINE

Estimated Development Duration: 10 Weeks

The timeline assumes approximately two developers, access to pilot data and timely WhatsApp Business onboarding.

Week	Development Phase	Expected Deliverables
1	Planning and project setup	Requirements, architecture, GitHub, Supabase, Meta onboarding
2	Authentication and company management	Login, company workspaces, roles, permissions
3	Property and tenant management	Properties, units and tenants
4	Lease management	Lease records, lease terms and document uploads
5	Statement processing	PDF uploads and Python extraction service
6	Statement validation	Manual review, matching and duplicate detection
7	Monitoring engine	Escalations, expiries, renewal deadlines, guarantees and arrears

Week	Development Phase	Expected Deliverables
8	WhatsApp integration	Company-specific numbers, templates, messaging and webhooks
9	Reporting and automation	Owner summaries, supported questions and arrangement reminders
10	Testing and pilot preparation	Security, integration, calculation testing and deployment

12.1 Week 1: Planning and Foundation

Activities:

- Confirm project requirements.
- Finalise technical architecture.
- Design initial database structure.
- Set up GitHub repository.
- Configure development environments.
- Create Supabase project.
- Create initial Next.js application.
- Obtain sample lease agreements and statements.
- Start Meta WhatsApp Business onboarding.

Deliverable: Development foundation ready.

12.2 Week 2: Authentication and Company Management

Activities:

- Implement authentication.
- Create company workspaces.
- Implement user roles.
- Configure permissions.
- Implement Row Level Security.
- Configure secure file storage.

Deliverable: Secure multi-company foundation.

12.3 Week 3: Property and Tenant Management

Activities:

- Develop property management.
- Develop unit management.
- Develop tenant management.
- Implement relationships between properties, units and tenants.

Deliverable: Functional property management module.

12.4 Week 4: Lease Management

Activities:

- Develop lease creation and editing.
- Capture rental amounts.
- Capture escalation information.
- Capture expiry dates.
- Capture renewal options.
- Capture deposit and guarantee information.
- Support lease document uploads.

Deliverable: Functional lease management module.

12.5 Week 5: Statement Processing

Activities:

- Implement PDF uploads.
- Develop Python extraction service.
- Extract relevant statement information.
- Store original documents.
- Return structured extraction results.

Deliverable: Initial automated statement extraction.

12.6 Week 6: Statement Validation

Activities:

- Implement data validation.
- Implement tenant matching.
- Detect duplicate statements.
- Build statement review screens.
- Allow manual correction.

Deliverable: Reliable statement processing workflow.

12.7 Week 7: Monitoring Engine

Activities:

- Implement escalation calculations.
- Implement expiry monitoring.
- Implement renewal deadline monitoring.
- Implement guarantee monitoring.

- Implement arrears detection.
- Generate monitoring events.

Deliverable: Core monitoring functionality.

12.8 Week 8: WhatsApp Integration

Activities:

- Integrate Meta WhatsApp Cloud API.
- Configure company-specific WhatsApp accounts.
- Implement message templates.
- Implement statement delivery.
- Implement payment reminders.
- Implement webhooks.
- Record delivery statuses.

Deliverable: Working WhatsApp integration using each client's own number.

12.9 Week 9: Reporting and Automation

Activities:

- Implement weekly owner summaries.
- Implement supported owner questions.
- Implement payment arrangement records.
- Implement instalment reminder scheduling.
- Complete message history.
- Implement job retries and error handling.

Deliverable: Automated reporting and communication.

12.10 Week 10: Testing and Deployment

Activities:

- Test financial calculations.
- Test company data isolation.
- Test document processing.
- Test WhatsApp sender selection.
- Test message approval.
- Test background jobs.
- Fix identified issues.
- Prepare pilot deployment.

Deliverable: MVP ready for controlled pilot use.

13. MVP TESTING AND ACCEPTANCE CRITERIA

The MVP must pass the following tests before being considered ready for pilot use.

13.1 Authentication and Access

- Users can log in securely.
- Users can access their authorised company workspace.
- Users cannot access unauthorised company records.
- User roles restrict access appropriately.

13.2 Property and Lease Management

- Users can create and edit properties.
- Users can create and edit units.
- Users can manage tenants.
- Users can create and update lease records.
- Lease information is correctly associated with the relevant tenant and unit.

13.3 Statement Processing

- PDF statements can be uploaded.
- Original documents are retained.
- Relevant information can be extracted.
- Uncertain information is flagged.
- Manual corrections can be made.
- Duplicate statements can be identified.
- Verified information is saved correctly.

13.4 Monitoring Accuracy

- Escalation calculations are accurate.
- Lease expiry dates are monitored correctly.
- Renewal deadlines are calculated correctly.
- Guarantee shortfalls are identified correctly.
- Arrears are identified from verified statement information.
- Monitoring events are traceable to their source data.

13.5 WhatsApp Functionality

- Each client uses its own WhatsApp Business number.
- Messages are sent using the correct company account.
- Approved templates are used where required.
- Statement attachments can be sent.
- Payment reminders can be scheduled.
- Owner summaries can be delivered.
- Incoming replies can be recorded.
- Delivery statuses are tracked.
- Duplicate webhook events are handled safely.
- Unapproved tenant messages are not sent.

13.6 Payment Restrictions

The system must not:

- Collect rental payments.
- Receive tenant funds.
- Process payments.
- Initiate bank transfers.
- Connect to banking systems to initiate transactions.
- Operate a payment gateway.
- Hold funds on behalf of clients.

All payments remain outside LeaseGuard.

14. PILOT IMPLEMENTATION PLAN

The first deployment should focus on one property.

The purpose is to validate the system before introducing it across a larger portfolio.

Phase 1: Data Collection

The pilot client provides:

- Three sample tenant statements.
- Relevant lease agreements.
- Property information.
- Unit information.
- Tenant contact information.
- Deposit and guarantee information.
- WhatsApp Business onboarding details.

Phase 2: Initial Setup

The development team loads the property information and captures the lease terms.

The sample statements are uploaded and processed.

Phase 3: Observation Mode

The monitoring engine runs without sending tenant messages.

The property manager reviews the findings.

The team validates:

- Statement extraction accuracy.
- Tenant matching.
- Rental escalation calculations.
- Lease expiry dates.
- Renewal deadlines.
- Guarantee calculations.
- Arrears information.

Phase 4: Controlled Communication

After the client approves the monitoring results:

- Configure the client's WhatsApp Business number.
- Complete required Meta approvals.
- Configure message templates.
- Confirm tenant contact information.
- Obtain required communication approvals.
- Enable messaging for a small group of tenants.

Phase 5: Expansion

Once the pilot has been successful, expand the system to additional properties and tenants.

15. SECURITY AND DATA PROTECTION

LeaseGuard will process commercially sensitive information.

The system must implement appropriate safeguards.

Requirements:

1. Company-level data isolation.
2. Role-based access control.
3. Secure authentication.
4. Protected document storage.

5. Encrypted sensitive credentials.
6. Secure API communication.
7. Input validation.
8. Audit logging.
9. Controlled access to tenant information.
10. Secure WhatsApp integration.
11. Appropriate message suppression and opt-out handling.
12. Compliance review against applicable South African data protection requirements, including POPIA.

16. MVP OUT OF SCOPE

The following features are explicitly excluded from the MVP.

Financial and Payment Functionality

- Payment collection.
- Payment processing.
- Payment gateway integration.
- Banking API integration.
- Bank transfers.
- Holding tenant funds.
- Payment reconciliation.
- Transaction management.
- Financial ledger functionality.

Other Excluded Functionality

- Replacing existing accounting systems.
- Issuing invoices.
- Calculating VAT.
- Direct integration with every accounting platform.
- Fully automated lease agreement interpretation.
- Automated renewal negotiations.
- Automated decisions about guarantee claims.
- Automated handling of disputed tenant charges.
- Native Android and iOS applications.
- Unrestricted AI chatbot functionality.

17. FUTURE ENHANCEMENTS

The following features may be considered after the MVP.

17.1 Advanced Document Processing

- More advanced PDF extraction.
- Automatic lease agreement extraction.
- Improved OCR.
- Additional accounting statement formats.
- Extraction confidence scoring.

17.2 Advanced Portfolio Reporting

- Portfolio-wide analytics.
- Historical arrears trends.
- Rental escalation performance.
- Guarantee exposure reports.
- Property comparisons.
- Financial risk indicators.

17.3 Additional Integrations

- Accounting software APIs where available.
- Email integration.
- Additional communication channels.
- Document management integrations.

17.4 Advanced Owner Assistant

- More detailed portfolio questions.
- Historical portfolio reporting.
- Property comparisons.
- Natural language reporting based on verified database information.

18. PROJECT RISKS AND MITIGATION

Risk	Impact	Mitigation
Different PDF layouts	Incorrect extraction	Client-specific extraction rules and manual review
Scanned documents	Missing information	OCR fallback and validation
Incorrect lease information	Incorrect monitoring	Human verification during onboarding
WhatsApp onboarding delays	Delayed communication features	Begin onboarding in Week 1
Incorrect WhatsApp sender	Messages sent from wrong company	Strict company-to-number mapping and automated tests
Duplicate background jobs	Duplicate reminders	Idempotency and job tracking
Incorrect calculations	Incorrect findings	Deterministic calculations and automated tests
Data exposure	Security and privacy risk	Row Level Security and access testing
Unapproved communication	Client relationship risk	Mandatory approval workflow

19. PROJECT DEPENDENCIES AND ASSUMPTIONS

The proposed timeline and scope depend on the following:

1. Availability of a pilot client.
2. Access to representative PDF statements.
3. Access to relevant lease agreements.
4. Availability of accurate tenant and property information.
5. Timely Meta WhatsApp Business onboarding.
6. Access to approved WhatsApp Business numbers.
7. Availability of message template approvals.
8. Timely feedback from the client during testing.
9. Availability of the proposed development resources.

The ten-week estimate assumes these dependencies are addressed without significant delays.

20. FINAL MVP DELIVERABLES

At the end of the MVP development period, the expected deliverables are:

1. A working LeaseGuard web application.
2. Secure multi-company authentication.
3. Property management module.
4. Unit management module.
5. Tenant management module.
6. Lease management module.
7. PDF statement upload functionality.
8. Automated statement extraction service.
9. Statement review and validation workflow.
10. Lease monitoring engine.
11. Rental escalation discrepancy detection.
12. Lease expiry and renewal deadline monitoring.
13. Deposit and bank guarantee monitoring.
14. Arrears monitoring.
15. Payment arrangement recording and reminder scheduling.
16. Company-specific WhatsApp Business integration.
17. Tenant statement delivery.
18. Automated tenant payment reminders.
19. Weekly owner WhatsApp summaries.
20. Supported owner WhatsApp questions.
21. Communication history and audit logs.
22. Testing and pilot deployment.

21. FINAL DEVELOPMENT RECOMMENDATION

LeaseGuard should be developed as a modular monolithic web application using Next.js, React, TypeScript, Supabase PostgreSQL and Python FastAPI for document processing.

Redis and BullMQ should manage background operations, while the official Meta WhatsApp Business Cloud API should handle all WhatsApp communication.

The application must support multiple client companies, each with its own approved WhatsApp Business number.

The MVP should prioritise accurate statement extraction, verified lease information, reliable monitoring rules and controlled WhatsApp communication.

No payment functionality should be developed or integrated.

LeaseGuard will remain a monitoring and communication layer that works alongside existing accounting software.

A ten-week development period is recommended, followed by a controlled pilot using one property before expanding to additional properties and clients.

> **The primary measure of MVP success is whether LeaseGuard can accurately identify lease and statement discrepancies, give property managers actionable information and reliably communicate approved payment reminders using the correct client's WhatsApp Business number.**